

The objective of this project was to build a **sentiment classification model** that can automatically determine whether a given text expresses a **positive or negative sentiment**. This was achieved using a **Long Short-Term Memory (LSTM)** neural network, a type of recurrent neural network (RNN) known for effectively capturing sequential dependencies in text data.

Dataset and Preprocessing

- **Dataset used:** IMDb Movie Reviews dataset (50,000 labeled movie reviews).
 - 25,000 reviews were used for training and 25,000 for testing.
 - Labels: 0 → Negative, 1 → Positive.
- **Data understanding:** Reviews were inspected for text patterns and class balance. Both sentiment classes were nearly equal, ensuring a balanced dataset.
- **Preprocessing steps:**
 1. Converted reviews into integer sequences using Keras's built-in IMDb tokenizer.
 2. Each sequence was **padded/truncated to 200 tokens** using `pad_sequences()` for uniform length.
 3. Vocabulary size limited to 10,000 most frequent words to reduce noise and complexity.

This preprocessing ensures the model receives clean, uniform numerical input sequences ready for deep learning.

Model Architecture

The model was built using **Keras Sequential API** with the following layers:

Layer	Description
Embedding Layer	Converts word indices to dense 128-dimensional word vectors (learned during training).
LSTM Layer (128 units)	Captures long-range dependencies in text sequences, learning contextual relationships between words. Dropout (0.2) and recurrent dropout (0.2) prevent overfitting.

Layer	Description
Dense Layer (Sigmoid activation)	Outputs a probability between 0 and 1, representing sentiment polarity (positive/negative).

Loss Function: Binary Cross-Entropy

Optimizer: Adam (learning rate = 0.001)

Epochs: 5

Batch Size: 64

Model Training and Evaluation

- Model was trained with **80% training** and **20% validation split**.
- Accuracy and loss were monitored across 5 epochs.
- After training, evaluation was performed on the **test set (25,000 samples)**.

Performance Metrics:

Metric	Score
Accuracy	79.84%
Precision (Positive)	0.81
Recall (Positive)	0.77
F1-Score (Positive)	0.79

Confusion Matrix:

The confusion matrix indicated balanced performance:

- True Negatives and True Positives were roughly equal.
- Slight misclassifications occurred for borderline reviews (e.g., “average movie” type comments).

Visualization:

- Accuracy and loss curves showed steady learning with minimal overfitting.
- Validation accuracy stabilized after 3–4 epochs, confirming model generalization.

Observations

- The LSTM model effectively learned **contextual sentiment patterns** (e.g., “not good” vs. “very good”).
- Validation and test accuracies were close, indicating good generalization.
- Training with 5 epochs provided optimal performance without overfitting.
- The model achieved an accuracy close to 80%, which is **strong for text sentiment classification using basic embeddings**.

Why LSTM was Chosen

- Traditional models like **Logistic Regression** or **Random Forest** rely on bag-of-words or TF-IDF features, which **ignore word order and context**.
- Sentiment often depends on **sequence structure** (e.g., “not bad” ≠ “bad”).
- LSTM networks can **remember long-term dependencies** through gating mechanisms, capturing contextual meaning over word sequences.
- Compared to basic RNNs, **LSTMs overcome vanishing gradient problems**, making them ideal for longer sentences or complex linguistic patterns.
- This makes LSTM a superior choice for natural language tasks like sentiment analysis.

Conclusion

In this project, an **LSTM-based neural network** was successfully implemented for binary sentiment classification on the IMDb dataset.

The model achieved **~80% accuracy**, showing strong capability in understanding textual emotion through sequence modeling.

This experiment demonstrates that **deep learning approaches (especially LSTM)** outperform traditional machine learning models by leveraging the **sequential nature of text data**.

Future improvements could include:

- Using **pre-trained embeddings (GloVe, Word2Vec, or BERT)** for richer semantic representation.

- Experimenting with **Bidirectional LSTM** or **GRU** layers for enhanced context capture.
- Expanding to **multi-class or multilingual sentiment analysis**.

Final Takeaway

“LSTMs enable the model to not just read words, but understand the *flow* of meaning — making them ideal for human language tasks like sentiment detection.”